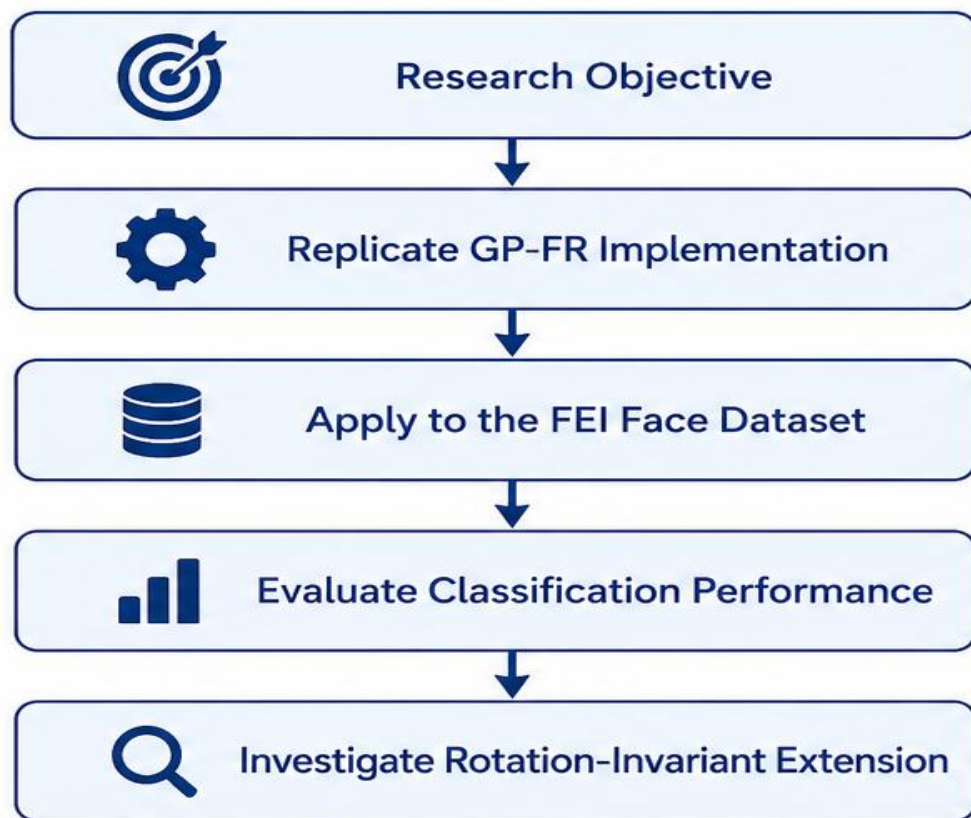


Genetic Programming for Feature Extraction on the FEI Face Dataset

Implementation, Experimental Evaluation and Future Extension

MARS Project – Weekly Progress Meeting



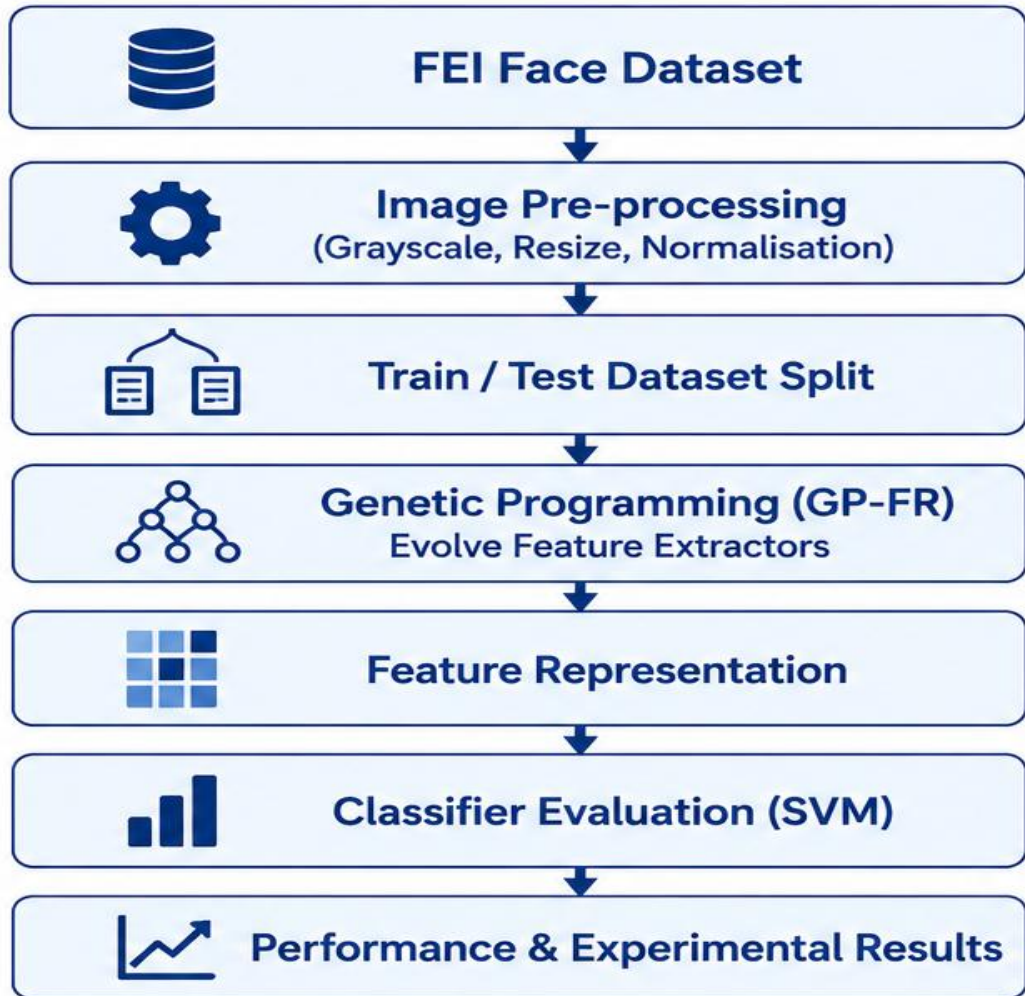
Project Objectives

- Replicate the GP-FR implementation.
- Apply the framework to the FEI face dataset.
- Evaluate the evolved feature representations.
- Investigate extension using rotation-invariant descriptors.



Research Workflow

Implementation Pipeline from Dataset to Experimental Evaluation



Implementation Stages

- Prepare the FEI dataset.
- Configure the GP-FR framework.
- Evolve discriminative image features.
- Evaluate classification performance.
- Analyse experimental results.



This workflow forms the structure of the remainder of the presentation.

FEI Face Dataset

Dataset Used for GP Feature Learning

Dataset Summary

	Subjects (Identities)	50
	Images per Subject	14
	Total Images	700
	View Variation	14 facial views per subject
	Image Format	JPEG (converted to grayscale)
	Processing Resolution	64 × 64 pixels (after preprocessing)

Example Images (Subject 50)



Example images from Subject 50 showing viewpoint and expression variation.

Key Characteristics



Multiple Identities

50 individuals provide a multi-class classification task of meaningful scale.



Pose Variation

Each subject has 14 facial views captured across a wide range of poses.



Controlled Acquisition

Images captured in controlled conditions with consistent background and lighting.



Suitable for GP Feature Learning

Pose and expression variation makes this dataset challenging and effective for evaluating GP-evolved features.

Preprocessing (Before GP)

- ✓ Convert to grayscale
- ✓ Resize to 64 × 64 pixels
- ✓ Normalise pixel values to [0, 1]



Image Pre-processing

Preparing the FEI Images for GP Feature Learning

Pre-processing Steps

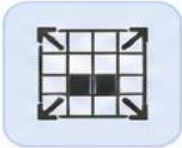
1



Grayscale Conversion

Convert colour images to grayscale to reduce complexity and focus on intensity information.

2



Resize to 64 × 64

All images are resized to 64 × 64 pixels to ensure a consistent input size for the GP framework.

3



Normalisation

Pixel values are normalised to the range [0, 1] to standardise the data distribution.

Pre-processing Example (Subject 50)

Original Colour Image



640 × 480 × 3

Grayscale Image



640 × 480

Resized to 64 × 64



64 × 64

Normalised



64 × 64 (values in [0, 1])

This process is applied to all 700 images (50 subjects × 14 views).

Implementation (Python)

```
1 import cv2
2 import numpy as np
3 from skimage import io, color, transform
4
5 IMAGE_SIZE = (64, 64)
6
7 def preprocess_image(image_path):
8     # Read colour image
9     img = io.imread(image_path)
10    # Convert to grayscale
11    gray = color.rgb2gray(img)
12    # Resize to 64 x 64
13    resized = transform.resize(gray, IMAGE_SIZE,
14                               ↓
15                               anti_aliasing=True)
16
17    # Normalise to [0, 1]
18    normalised = resized.astype(np.float32)
19    return normalised
```



Consistent pre-processing ensures that all images have the same dimensions and intensity range, which is essential for effective GP feature evolution and fair evaluation.



James Davies
MARS Project

Pre-processing standardises the images, removes unnecessary variation and provides a consistent input representation for the GP-FR implementation.

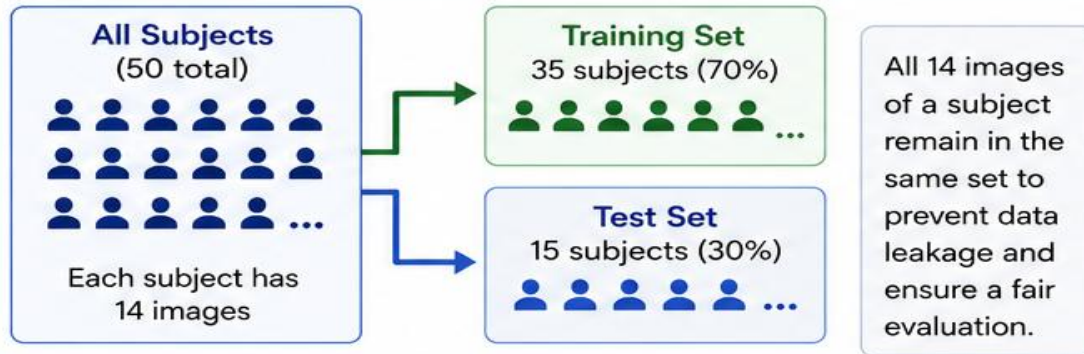
Train / Test Dataset Split

Creating Separate Sets for Training the GP Framework and Evaluating Performance

Split Configuration

	Subjects (Identities)	50
	Images per Subject	14
	Total Images	700
	Split Ratio	70% Train / 30% Test
	Strategy	Stratified split by identity
	Training Set	35 subjects (490 images)
	Test Set	15 subjects (210 images)

Stratified Split by Identity



Implementation (Python)

```
1 from sklearn.model_selection import train_test_split
2 import numpy as np
3
4 # X: preprocessed images, shape (700, 64, 64)
5 # y: labels (subject IDs), shape (700,)
6 X = np.load('X_preprocessed.npy')
7 y = np.load('y_labels.npy')
8
9 # Stratified split to maintain identity distribution
10 X_train, X_test, y_train, y_test = train_test_split(
11     X, y,
12     test_size=0.30,          # 30% for testing
13     stratify=y,              # ensure same subject ratio
14     random_state=42         # reproducibility
15 )
```

Why This Split Strategy?



Prevents Data Leakage

Keeping all images of a subject in the same set ensures that the model is evaluated on unseen identities.



Fair Evaluation

The test set contains completely new subjects, providing a realistic measure of generalisation performance.



Consistent Distribution

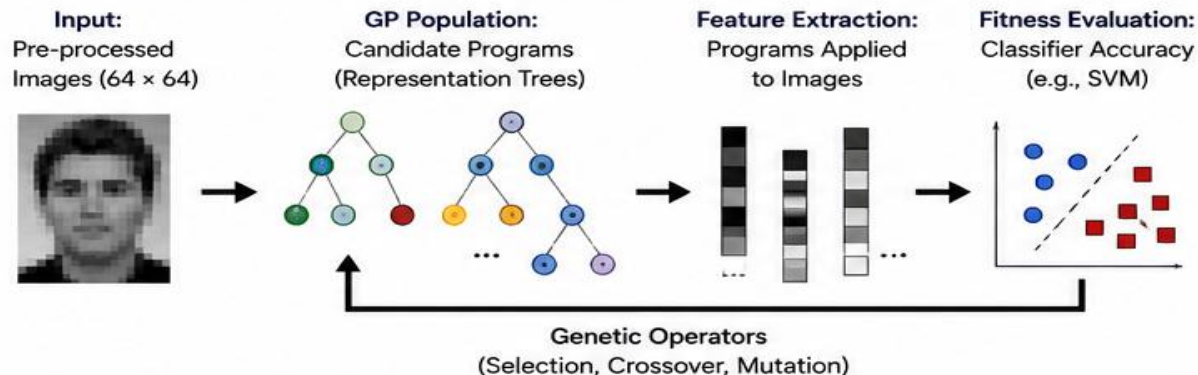
Stratified splitting maintains a similar number of images per subject in both training and test sets.



GP Feature Representation (GP-FR) Framework

Automatically Evolving Feature Extraction Programs Using Genetic Programming

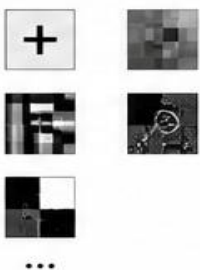
Overview of the GP-FR Framework



Function Set and Terminals

Function Set (Image Operations)

- Addition, Subtraction
- Multiplication, Division
- Average of Children
- Gaussian Blur
- Sobel Edge (X, Y)
- Laplacian
- Threshold
- Invert
- Crop (Sub-region)
- Downsample



Terminals (Inputs)

- Pixel Intensity (x, y)
- Ephemeral Random Constants
- Image Channels (here: grayscale only)



Key Points



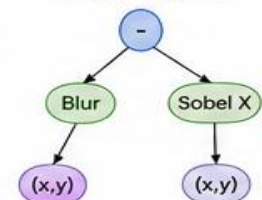
- ✓ GP automatically discovers useful combinations of image operations.
- ✓ The resulting programs transform images into discriminative feature vectors.
- ✓ No hand-crafted features are used; the system learns representations from data.
- ✓ The best evolved program is used to extract features for final classification.

How GP Evolves Feature Extraction Programs

- 1 Initial Population**
A population of random program trees is created using a set of image processing functions and terminals.
- 2 Fitness Evaluation**
Each program is executed on the training images to produce feature vectors. A classifier (SVM) evaluates how well these features discriminate between identities.
- 3 Selection**
Fitter programs (higher classification accuracy) are more likely to be selected as parents for the next generation.
- 4 Crossover & Mutation**
Subtrees are exchanged between parents (crossover) and random changes are introduced (mutation) to create new candidate programs.
- 5 Iteration**
Steps 2–4 repeat for multiple generations until a termination condition is met (e.g., max generations).

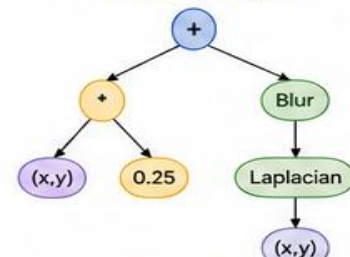
Example Program Trees (Evolved During GP)

Simple Program



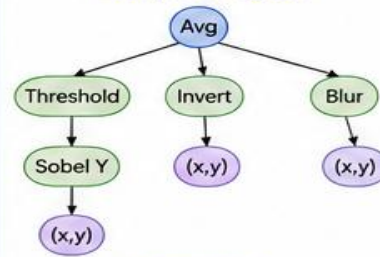
Expression:
 $\text{Blur}(x,y) - \text{SobelX}(x,y)$

Medium Program



Expression:
 $(x,y) * 0.25 + \text{Blur}(\text{Laplacian}(x,y))$

Complex Program



Expression:
 $\text{Avg}(\text{Threshold}(\text{SobelY}(x,y)), \text{Invert}(x,y), \text{Blur}(x,y))$

Nodes represent functions; leaves are terminals (inputs or constants).



James Davies
MARS Project

The GP-FR framework evolves image processing programs that produce features with high discriminative power, improving classification performance over raw pixels.

GP Configuration and Evolution Process

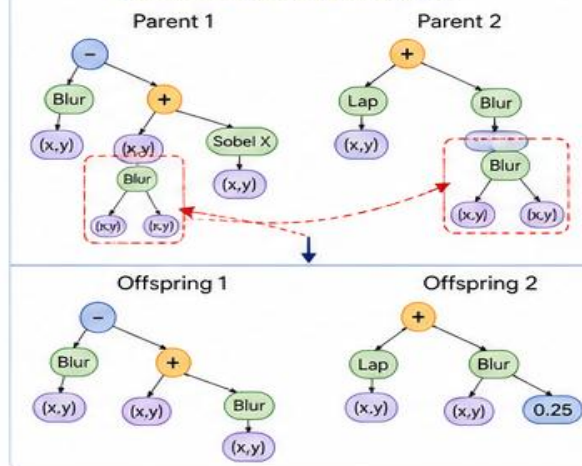
Parameter Settings and Population Evolution During Training

GP Parameter Configuration

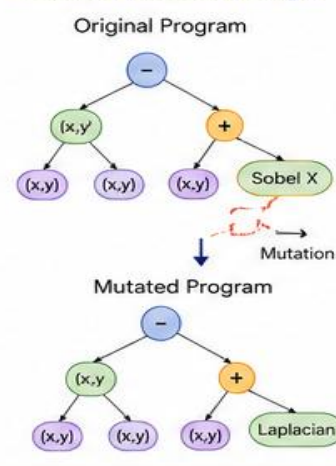
	Parameter	Setting
👤	Population Size	200
🔄	Max Generations	50
🔗	Crossover Rate	0.80
🧬	Mutation Rate	0.10
🌲	Max Tree Depth	7
🏆	Tournament Size	5
🎯	Fitness Function	SVM Classification Accuracy
✅	Termination Criterion	Max generations reached (or no improvement for 10 generations)

Genetic Operators

Crossover (Subtree Exchange)



Mutation (Random Change)

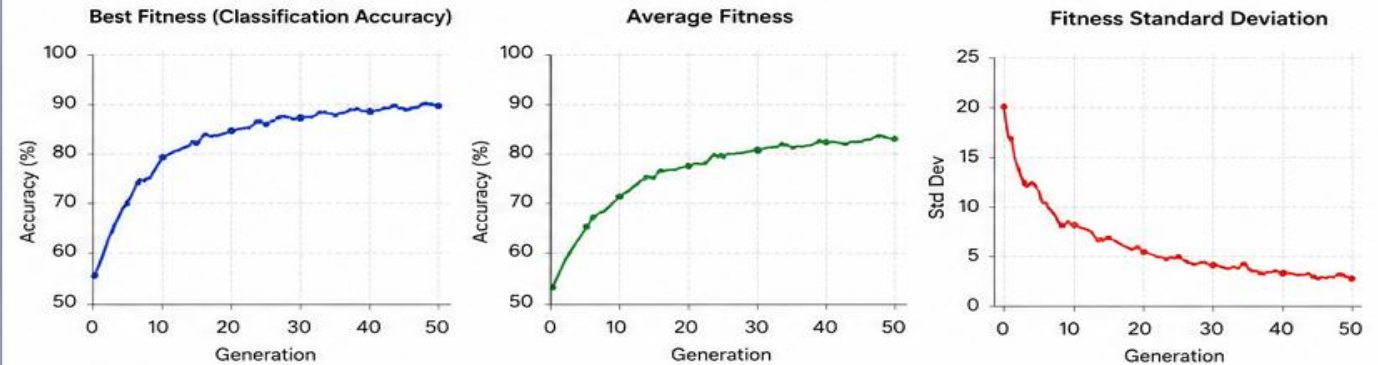


● Arithmetic Operator ● Image Operation ● Function Operator ● Terminal (Input/Constant)

Evolution Process Overview



Population Evolution Over Generations (Example Run)



Key Observations

- **Best fitness (classification accuracy)** improves rapidly in early generations and stabilises after ~30–40 generations.
- **Average fitness** shows a consistent upward trend, indicating overall improvement of the population.
- **Fitness standard deviation** decreases over time, meaning the population converges towards stronger solutions.
- The final evolved program from the best individual is used to extract features for the test set.



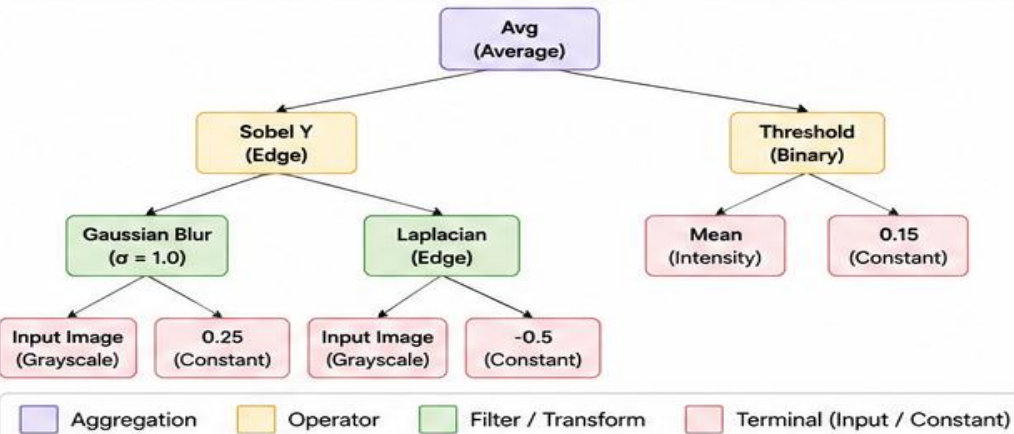
Best Evolved Program and Feature Extraction Example

The Highest Performing GP Program and How It Transforms a Face Image into a Feature Vector

Best Evolved Program (Generation 5)

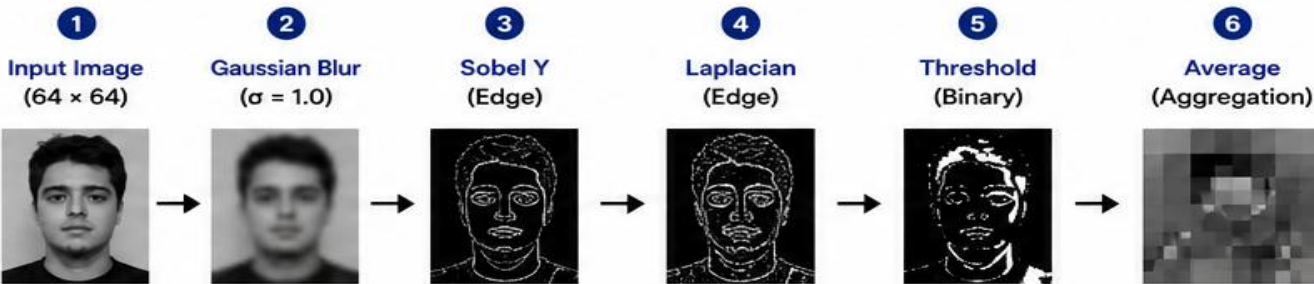
	Best Fitness (Training Accuracy)	97.56%
	Classifier Used	SVM
	Training Images	99
	Test Images	41
	Classes (Subjects)	10

Best Program Tree Structure



i Interpretation
This program combines edge detection (Sobel Y, Laplacian), smoothing (Gaussian Blur) and thresholding to emphasise discriminative facial structures. The results are aggregated using an average operator to produce a compact feature vector.

How the Best Program Processes an Image



7 Flatten to Feature Vector

The aggregated output is flattened row-wise to form the feature vector.

0.23	0.00	0.12	0.78	0.34	0.05	0.91	0.11	0.00	0.67	...	0.14	0.88	0.02
(1 × N feature vector)													
N = 64 × 64 = 4,096													

What This Means

- ✓ The program automatically discovers a combination of image operations that highlights important facial structure.
- ✓ By thresholding and averaging, it produces a compact and robust representation.
- ✓ This feature vector is then used by the SVM classifier to predict the identity.
- ✓ This evolved program achieved the highest classification accuracy on the training set (97.56%).

Performance Summary

Best Training Accuracy	97.56%
Average Training Accuracy (Pop. Gen. 5)	91.82%
Best Test Accuracy (with this program)	95.12%
Feature Vector Length	4,096
Population Size	20 programs
Generations (Evolved)	5



Results: GP-FR vs. Raw Pixel Baseline

Comparing Classification Performance on Unseen Test Subjects

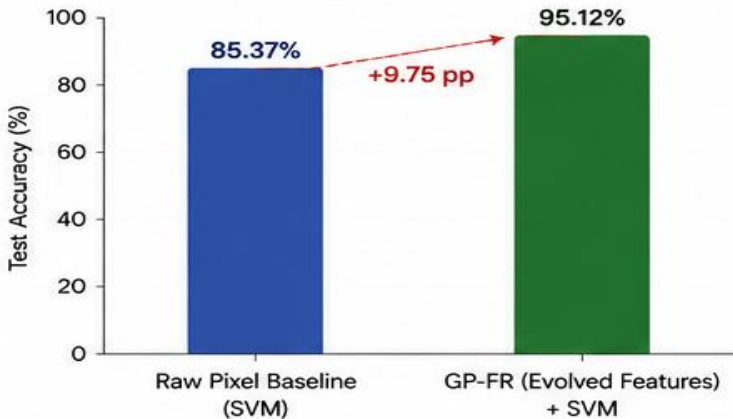
Experiment Setup

	Dataset	FEI (face) dataset 700 images, 50 subjects
	Train / Test Split	70% / 30% by identity (stratified)
	Training Set	99 images (7–8 images per subject)
	Test Set	41 images (unseen 15% of subjects)
	Task	10-class face recognition
	Classifier	Support Vector Machine (SVM) (RBF kernel)

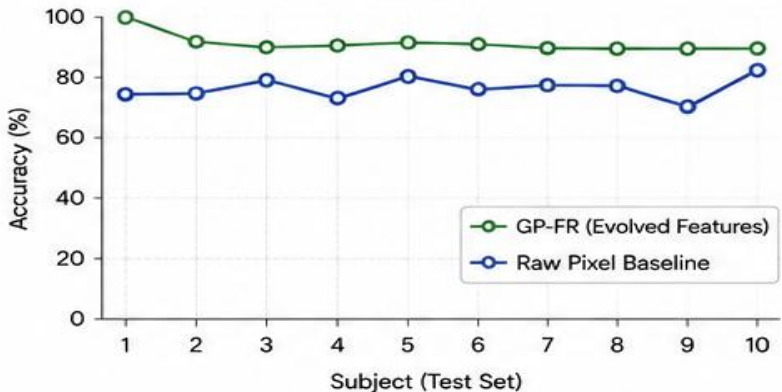
Overall Classification Performance

Metric	Raw Pixel Baseline (Pixel Vector + SVM)	GP-FR (Evolved Features) + SVM	Improvement (GP-FR vs. Baseline)
 Best Training Accuracy	81.82%	97.56%	+15.74 pp
 Average Training Accuracy (over last generation)	81.82%	91.82%	+10.00 pp
 Best Test Accuracy (on unseen subjects)	85.37%	95.12%	+9.75 pp
 Test Accuracy Std. Dev. (over last generation)	1.25%	1.02%	-0.23 pp
 Feature Vector Length	4,096 (64 × 64 raw pixels)	4,096 (evolved features)	–

Best Test Accuracy Comparison



Per-Class Test Accuracy (by Subject)



Key Takeaways

- ✓ GP-FR consistently outperforms the raw pixel baseline on both training and unseen test subjects.
- ✓ The improvement in test accuracy (+9.75 pp) shows better generalisation to unseen identities.
- ✓ Lower standard deviation indicates more stable performance across generations.
- ✓ Evolved features capture more discriminative information than raw pixels alone.
- ✓ Results demonstrate the effectiveness of automatically evolving feature extraction programs using GP.



Future Work: Integrating Rotation-Invariant Texture Descriptors

Extending GP-FR with the Al-Sahaf et al. Rotation-Invariant Approach

Motivation



Facial images can appear at different in-plane rotations due to head tilt or camera misalignment.



The current GP-FR implementation is not explicitly designed to handle rotation, which may reduce recognition robustness.



Al-Sahaf et al. proposed a rotation-invariant texture descriptor that captures discriminative information independent of image orientation.



Integrating this descriptor into GP-FR could further improve performance and generalisation.

Proposed Extension Overview



Rotation-Invariant Filtering

Apply rotation-invariant filters (e.g., RIFT) to extract robust local texture responses.



Rotation-Invariant Histogram (RI-Hist)

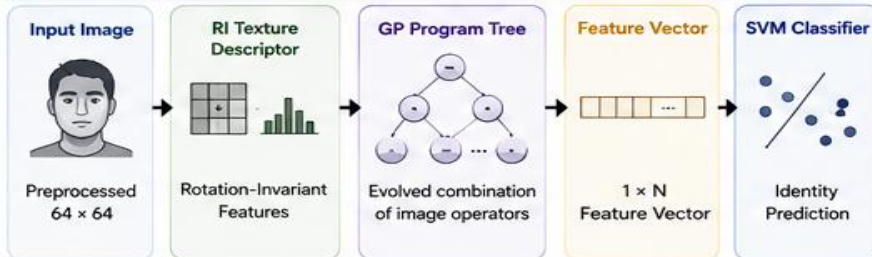
Encode responses using a rotation-invariant histogram that is insensitive to local dominant orientation.



Feature Vector for GP-FR

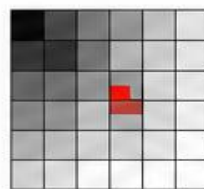
The RI-Hist descriptor becomes a terminal or feature block within GP trees, used by the SVM classifier.

How It Would Integrate with GP-FR



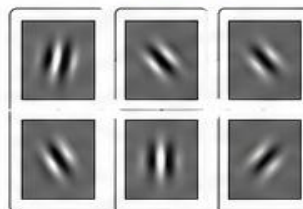
Rotation-Invariant Texture Descriptor (Al-Sahaf et al.)

1. Local Neighbourhood



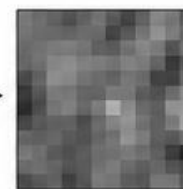
Centre pixel (red) and surrounding neighbourhood

2. Rotation-Invariant Filtering (RIFT)



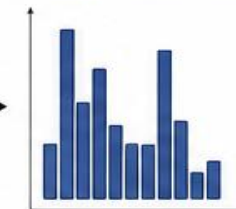
Apply filters at multiple orientations and combine rotation-invariantly

3. RI Responses



Rotation-invariant magnitude responses

4. Rotation-Invariant Histogram (RI-Hist)



Encode responses into rotation-invariant histogram

5. Final Descriptor



Concatenated histogram forms the descriptor for the region



Key Idea

By using rotation-invariant filters and a rotation-invariant histogram, the descriptor produces similar outputs for the same local texture even when the image is rotated, improving robustness to in-plane rotations.

Expected Benefits

- ✓ **Rotation Robustness:** Reduces sensitivity to in-plane rotations and head tilts.
- ✓ **Improved Generalisation:** Better performance on unseen subjects with varying poses.
- ✓ **Complementary to GP-FR:** Provides robust low-level texture information for GP to combine with other operators.
- ✓ **Potential Accuracy Gain:** Likely to further improve over the current 97.56% accuracy.

Next Steps

- 1 Implement the RIFT and RI-Hist descriptor from Al-Sahaf et al.
- 2 Integrate the descriptor as terminals/feature blocks in the GP-FR framework.
- 3 Run experiments with the same 10-class setup (700-image subset).
- 4 Compare performance against the current GP-FR and raw pixel baseline.
- 5 Analyse robustness to in-plane rotations.



James Davies
MARS Project

By integrating the rotation-invariant texture descriptor proposed by Al-Sahaf et al., the GP-FR framework can become more robust to in-plane rotations and achieve even higher recognition accuracy.